实验三 中断与轮询（Interrupt vs Polling）

一、实验目的

完成本次实验之后，学生应该能够：

1. 理解轮询（polling）与中断驱动（interrupt-driven）两种处理外设事件方式的基本思想、优缺点与适用场景。

2. 掌握 RISC-V（M-mode）中断基本机制：中断源、mcause、mtvec、陷入（trap）与返回（MRET），理解 direct/vectored 向量模式。

3. 能够在 DDR200T + HBirdv2 E203 上实现基于中断的简单应用：

   (1) UART0 接收字符触发中断并控制 LED

   (2) 按键触发 GPIOA 中断并上报事件/翻转 LED

4. 建立“CPU（CSR/Trap）-> 中断控制器（PLIC）->外设（UART/GPIO）”的系统视角，理解“为什么必须 claim/complete”与“为什么必须清 pending/状态位”。

二、预习要求

1．阅读教材 Stalling (10th) 关于 I/O、中断、异常/陷入机制与 I/O 编程模型章节（轮询 vs 中断驱动的权衡）。

2．阅读 RISC-V Privileged Spec（至少覆盖）：

（1）mtvec：Direct/Vectored 模式与 BASE/4xcause 规则

（2）MRET/WFI：tarp return 的状态位恢复与 WFI 行为（可用于中断驱动“空闲等待”）

（3）mcause：机器外部/定时器/软件中断等 cause 编码（关注 Machine External Interrupt / Machine Timer Interrupt）

3．查阅 HBirdv2 SoC 外设文档：

（1）PLIC：中断源编号、优先级、enable、threshold、claim/complete 流程

（2）GPIOA Interrupt：INTEN/INTTYPE0/INTTYPE1/INTSTATUS 的含义与清除方式（读 INTSTATUS 清除）

（3）UART0：寄存器 offset（RBR/THR/IER/IIR/FCR/LSR）、IER 位定义与 IIR 中断原因码

三、实验环境与准备

1．硬件：Nuclei DDR200T FPGA Evaluation Kit（HBirdv2 E203 SoC bitstream 已加载）

2．软件：Nuclei Studio + OpenOCD + GDB；串口终端（UART0）

3．参考：HBirdv2 SoC peripherals 文档（UART/PLIC/GPIO/CLINT）

四、实验内容与任务概览

1．任务 1：轮询版本（Polling Baseline）：实现一个“可工作的外设事件处理”，但完全依赖轮询，作为对照组。

（1）UART0 轮询：读 UART_LSR.DR（bit0）判断是否收到字符；收到后回显并控制 LED。

（2）GPIO 轮询：周期性读取 GPIO_PADIN，检测按键状态变化（上/下沿由软件比较实现）。

（3）记录：CPU 忙等程度、响应延迟。

2．任务 2：搭建最小 Trap 框架（RISC-V 中断入口/返回）：完成“能进中断、能安全返回”的最小闭环。

（1）设置 metvec（先用 Direct 模式最简单：所有 trap 进同一个入口）。

（2）在 trap handler 中读取 mcause 区分：Machine External Interrupt（外部中断）/Machine Timer Interrupt（可选）等。

（3）使用 MRET 返回，理解其对 mstatus.MIE/MPIE 等字段的恢复动作。

3．任务 3：UART0 接收中断（Interrupt-driven UART RX）：用中断代替“轮询 LSR.DR”。

（1）UART0 寄存器与 offset：

IER offset=0x04；启用 RX watermark interrupt: ERXTHI 位（bit0）。

IIR offset=0x08; 可判断当前中断原因（RX watermark=8 等）。

（2）PLIC：UART0 的 interrupt source ID = 3；外设中断最终汇聚为 Machine External Interrupt。

（3）ISR（中断服务程序）建议：只做“快路径”

claim 得到 IRQ ID -> 读取 UART 数据（可 while（LSR.DR）清空 FIFO）->设置 flag/写 ring buffer->complete。

五、实验步骤

1．实验环境与工程准备

（1）启动 NucleiStudio，新建工程（Board 选择 DDR200T，SoC 选择 HBirdV2/E203，下载方式按工程模板使用 ILM）。

（2）确认 Terminal 窗口已经配置就绪：使用开发板对应的 UART0 调试串口，波特率/数据位/校验位按 BSP/工程默认配置一致。

（3）Clean 后 Build 工程，确保基础工程可编译链接通过，再进入开发。

2．任务 1：MMIO 映射与轮询访问（Polling）

（1）建立 MMIO 访问抽象

新建 application/lab3_mmio.h（或同等文件），实现：

static inline void mmio_write32(uint32_t addr, uint32_t val)

static inline uint32_t mmio_read32(uint32_t addr)

采用 volatile 指针访问，保证不会被编译器优化掉（寄存器读写语义保留）。

（2）整理外设地址与寄存器偏移（写入实验报告“地址映射表”）

根据 HBirdV2 外设文档确认：GPIOA、UART0 的基地址与寄存器偏移（用于后续任务复用）。

UART0 基地址与常用寄存器偏移（RBR/THR、IER、IIR\FCR、LSR 等）。

GPIOA 基地址与 PADDIR、PADOUT、INTEN/INTTYPE/INSTATUS 等寄存器定义。

（3）GPIO（LED）轮询式控制验证

GPIOA：配置 LED 对应引脚为输出（写 PADDIR），循环内周期性翻转 PADOUT（或按板级定义的 LED mask 操作）。

观察开发板 LED 是否按预期闪烁，作为 MMIO 写入验证闭环。

（4）UART0 轮询式输出验证

UART0：轮询 LSR 的 THRE/TEMT（或 SDK 提供的发送就绪位）后写 THR 发送字符。

在串口终端看到稳定输出（如启动 banner、循环计数等），作为 MMIO + UART 输出验证闭环。

（5）任务 1 验收（自检）

LED 按规律翻转：串口持续打印（无卡死/乱码/丢字）。

在报告中记录关键寄存器、地址与现象截图。

3．任务 2：Trap/中断框架 + 软件中断（MSI）触发与交互

（1）建立 trap 入口汇编文件

新建 application/lab3_trap_entry.S（注意使用大写.S，确保支持预处理与标准汇编指

令/伪指令）。

在 lab3_trap_entry.S 中实现：

保存必要通用寄存器现场（至少 caller-save；建议按统一栈帧保存全寄存器以简化）；

读取 mcause/mtval/mepc 等关键信息；

调用 C 函数（如 lab3_trap_handler()）；

恢复寄存器并 mret 返回。

导出符号 lab3_trap_entry（.global lab3_trap_entry），确保链接器可见。

（2）实现 trap 初始化与 MSI 触发接口

新建/完善 application/lab3_trap.c，实现：

lab3_trap_init()：设置 mtvec = lab3_trap_entry，并在 mstatus/mie 中打开 Machine 全局中断与软件中断使能。

lab3_trigger_msi()：向 CLINT 的 msip 写 1 触发软件中断；在 ISR 中清除 msip。

CLINT 基地址与 msip 机制参考 HBirdV2 文档。

定义全局计数器（如 volatile uint32_t g_msi_count），在软件中断 ISR 中自增，用于主循环观察。

（3）主循环做“可交互触发”

在 main.c：

启动时调用 lab3_trap_init()；

提供串口交互命令（例如输入 t 或回车）后调用 lab3_trigger_msi()；

主循环打印 g_msi_count 的变化或打印“MSI happened”事件日志。

通过串口输入触发后，观察计数器变化与日志输出，证明：UART 输入->程序触发->中断进入->ISR 执行->返回主循环链路闭环。

（4）任务 2 验收（自检）

任意多次触发 MSI，系统不死机；g_msi_count 单调递增；串口日志与触发次数可一致（可截图）。

4．任务 3：UART0 接收中断（Interrupt-driven UART RX，替代轮询 LSR.DR）

（1）UART0：开启接收中断源

按 UART 寄存器定义：

配置 FIFO（如需要）；

使能 IER 中的 RX 可用中断（Received Data Available）。

说明：具体中断原因可通过 IIR 判别；数据读取通过 RBR，数据就绪通过 LSR.DR 判别（但此处仅在 ISR 内使用，不再在主循环轮询）。

（2）PLIC：配置 UART0 外部中断路由

根据 HBirdV2 文档确认：PLIC 基址与 UART0 对应的中断源 ID（例如 UART0 = 3）。

初始化步骤（按 PLIC 标准流程）：

设置 UART0 中断源优先级（Priority > 0）；

在 Hart0 的 Enable 位图中使能该中断源；

设置 Hart0 的 Threshold（通常设 0 允许所有优先级）；

在 mie/mstatus 中开启 Machine External Interrupt（MEIE）与全局中断。

（3）在 trap handler 中接入 PLIC claim/complete

在 C 级 trap handler 中识别“外部中断”类型后：

读取 PLIC claim/complete 寄存器得到 irq_id；

若 irq_id == UART0_IRQ_ID：

循环读取 UART0：当 LSR.DR==1 时读 RBR 取字节（把“轮询”限定在 ISR 内部，用于一次性 drain FIFO/缓冲，避免丢字）；

将字节写入软件环形缓冲区/队列，或直接回显并生成事件日志；

最后向 claim/complete 写回 irq_id 完成中断。

（4）主循环：从“轮询寄存器”改为“消费缓冲区/事件

主循环不再读取 UART0->LSR.DR；改为：

检查 RX 环形缓冲区是否有数据（纯内存变量轮询或使用事件标志）；

取出数据做命令解析/回显/触发任务 2 的 MSI 等交互动作。

（5）任务 3 验收（自检）

不输入串口时，主循环不做 UART 寄存器轮询也能保持运行；

输入字符后立即被 ISR 捕获并回显/打印“RX IRQ”事件；

快速连续输入不丢字（ISR 内 drain + 缓冲区容量合理）；

报告中给出：PLIC 配置要点、UART IER/IIR/LSR 使用说明、现象截图。

六、实验记录与报告要求

1．MMIO 映射表（必填）：至少包含

（1）UART0 base 与寄存器 offset（RBR/THR/IER/IIR/FCR/LSR）

Peripheral	Base	Reg	Offset	R/W	用途	代码宏名
UART0	0x1001_3000	RBR	0x00	R	RX 数据读取	...
...	...	...	...	...	...	..

（2）GPIOA base 与寄存器 offset（PADDIR/PADIN/PADOUT/IOFCFG/INTEN/INTTYPE0/INTSTATUS）

Peripheral	Base	Reg	Offset	R/W	用途	代码宏名
GPIOA	0x1001_2000	PADDIR	0x00	R/W	LED 输出/按键输入方向	...
...	...	...	...	...	...	...

（3）PLIC：priority、enable、threshold、claim/complete 关键地址与解释；UART0 = 3、GPIOA = 15

项	值	用途
UART0 PLIC source ID	3	在 ISR 中区分来源
GPIOA PLIC source ID	15	同上
PLIC enable base (Target0)	0x0C00_2000	打开 source 3/15
...	....	...

2．两套实现（Polling vs Interrupt）的关键代码片段与运行现象截图/日志。

统一用“索引表+代码片段截图/摘录”

功能点	文件	函数/符号	行号范围	说明
UART 轮询接收	main.c	poll_uart_rx()	120-180	轮询 LSR.DR 后读 RBR
…	…	…	…	…

表 1 关键代码索引（Polling）

功能点	文件	函数/符号	行号范围	说明
trap 入口	trap.S	trap_entry	1-80	保存现场，读 mcause
…	…	…	…	…

表 2 关键代码索引（Interrupt）

3．“中断处理流程图”：触发源 -> PLIC -> MEI -> trap_entry -> claim -> ISR -> complete -> MRET。可用 draw.io/Visio/PowerPoint 画，或用 mermaid。

4．对比数据：轮询周期、平均/最大响应延迟、CPU 忙等比例。

（1）参数声明

参数	轮询（Polling）	中断（Interrupt）
UART 测试字符数 N		
GPIO 按键触发次数 N		
轮询周期 Tpoll（ms）		

（2）响应延迟

- Polling：从“主循环开始一次轮询检查（读状态寄存器）”到“完成本次事件处理（LED 更新/打印完成）”的时间
- Interrupt：从“进入 ISR（trap_entry 内确认 MEI 后）”到“完成本次事件处理并 complete”的时间

（3）数据表

指标	Polling 平均	Polling 最大	Interrupt 平均	Interrupt 最大
UART 处理开销（cycle 或 us）				

GPIO 处理开销（cycle 或 us）				

（4）CPU 忙等比例

学生必须在代码中统计两个累积量：

cycles_total：测试窗口总 cycle

cycles_idle：轮询模式为 delay/空转循环累积；中断模式为 WFI/空转等待积累

并按固定公式填写：

busy_wait_ratio = cycles_idle / cycles_total

表格固定为：

模式	cycles_total	cycles_idle	busy_wait_ratio
polling			
interrupt			

七、思考题

1．为什么 PLIC 必须 claim/complete？如果只读不写 complete，会出现什么现象？

2．GPIO 中断为什么“读 INSTATUS 就能清除 pending”？这种“读清”机制有什么优点/缺点？

3．UART 中断为什么需要 IER 使能，同时还要在 ISR 中读取数据（可能循环读空 FIFO）？如果只读一次会怎么样？

4．mtvec 的 Direct 与 Vectored 模式有什么取舍？Vectored 模式下入口地址如何计算？

5．为什么不建议在 ISR 里做大量 printf 或长延时？你会如何设计“ISR 快路径+主循环慢路径”？（提示：flag/ring buffer）

八、附录

附录 A 关键硬件资源速查表（HBirdv2 SoC）

A.1 外设基地址与中断号（与本实验任务 1－3 直接相关）

UART0 基地址：0x1001_3000

GPIOA 基地址：0x1001_2000（GPIOB：0x1004_0000）

外部中断号（PLIC Source ID）：UART0=3、GPIOA=15

附录 B UART0（16550 风格）寄存器与常用位

B.1 UART0 寄存器偏移（任务 1：轮询；任务 3：RX 中断）

寄存器	偏移	说明
RBR	0x00	接收缓冲寄存器（读）
THR	0x00	发送保持寄存器（写）
IER	0x04	中断使能寄存器
IIR	0x08	中断识别寄存器（读）
FCR	0x08	FIFO 控制寄存器（写）
LCR	0x0C	线路控制寄存器
LSR	0x14	线路状态寄存器

以上为 HBirdv2 SoC UART 寄存器定义与偏移（UART0 基址见附录 A）。

B.2 轮询/中断常用位

LSR.DR（bit0）：接收数据就绪（轮询替代对象）

LSR.THRE（bit5）：THR 空，可写发送（用于 putc/printf 的底层发送等待）

注：IER/IIR/FCR 的位定义在不同实现上可能略有扩展；本实验只需启用“RX 数据可用中断”和在 ISR 中通过 IIR/LSR/RBR 完成接收即可。寄存器地址/偏移以 HBirdv2 文档为准。

附录 C PLIC 关键信息

C.1 PLIC 外部中断源列表（节选）

Interrupt Number	Source
3	uart0
15	gpioA

（UART0 RX 中断属于外部中断源，经 PLIC 仲裁后以 Machine External Interrupt 进入内核。）

C.2 PLIC 关键寄存器地址（单核 target0）

寄存器/区域	地址（或起始地址）	说明
Source n priority	0x0C00_0004 起	每个 source 的优先级（n 从 1 开始）
Pending array	0x0C00_1000 起	pending 位图（只读）
Target0 enables	0x0C00_2000	target0 使能位图
Target0 threshold	0x0C20_0000	优先级阈值
Target0 claim/complete	0x0C20_0004	读 claim / 写 complete

附录 D CLINT 软件中断要点

CLINT 提供 msip 位用于触发/清除软件中断：写 1 触发、写 0 清除，仅 bit0 有效。

CLINT 同时提供 mtime/mtimecmp 以支持定时器中断

附录 E 常见构建/链接问题记录

1. undefined reference to lab3_trap_entry

典型原因：lab3_trap_entry.S 未参与编译或符号未导出（缺少 .global lab3_trap_entry），或 C 侧声明与符号名不一致。

处理建议：确认 application/subdir.mk 中包含该 .S；入口符号全局可见；C 侧 extern

void lab3_trap_entry(void);。

2. Assembler messages: unknown CSR csr

典型原因：内联汇编把 CSR 名写成了占位符（例如 "csrw csr, %0"），而不是合法 CSR（如 mtvec/mstatus/mie）。

处理建议：改为显式 CSR 名，或使用 NMSIS/SDK 提供的 CSR 访问宏（若工程已集成）。

3. OpenOCD JTAG scan chain / Unsupported DTM version

典型原因：接口/驱动选择、目标供电、JTAG 时钟或配置与板卡不匹配。

处理建议：优先核对板卡供电与 JTAG 连接；降低 adapter speed；确保所用 OpenOCD/配置脚本与目标一致；重启 NulceiStudio，重启板卡。